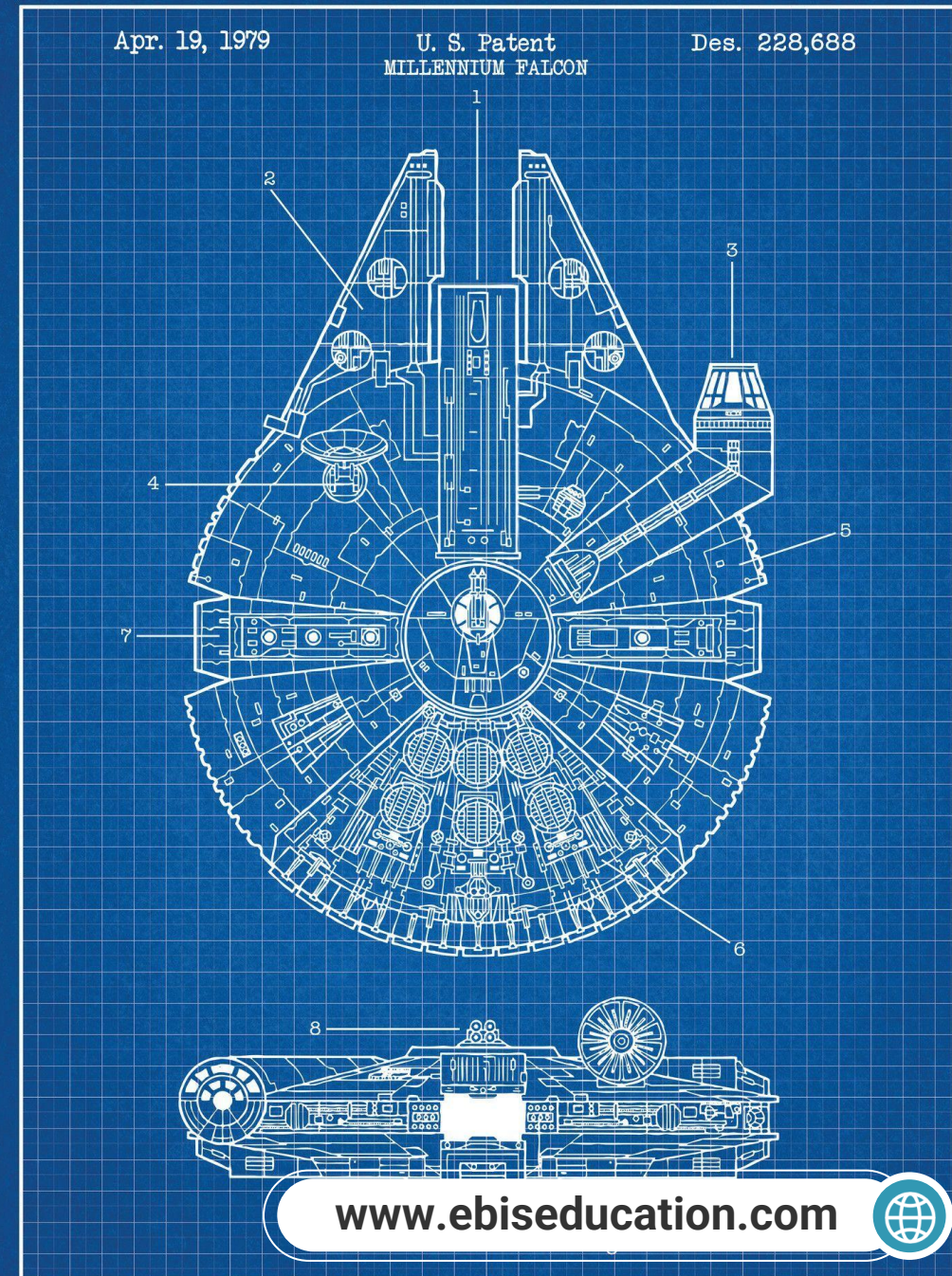


Gestión de proyectos



- Idea
- Análisis
- Planificación
- Ejecución
- Entrega
- Mantenimiento



Idea



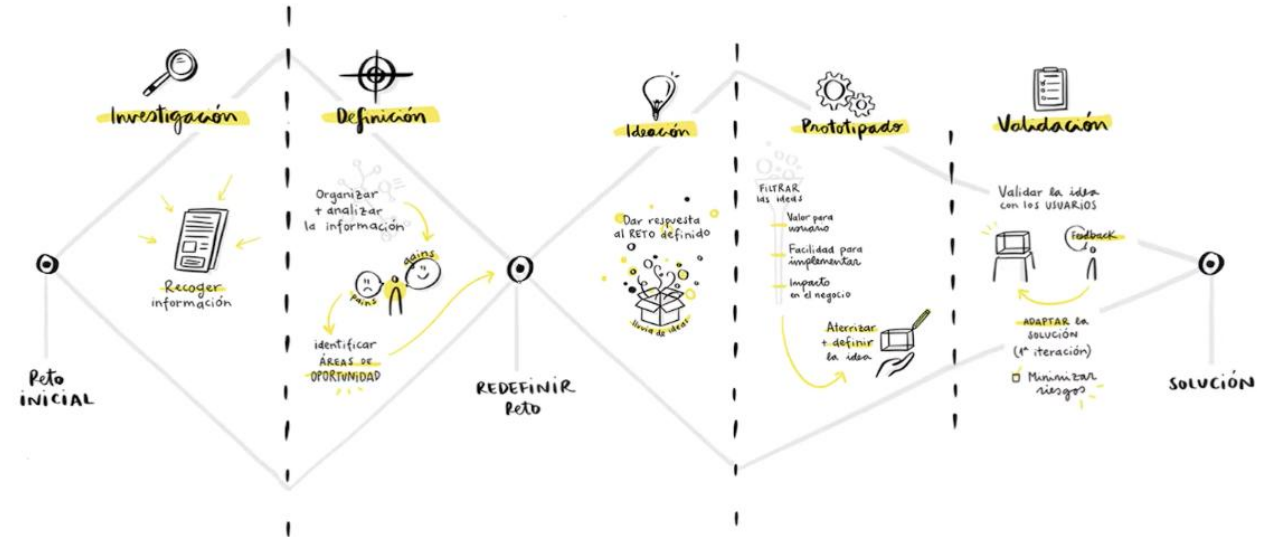
Idea

- Introspección
- Skills previos
- Unir puntos
- Bajar a tierra
- Project plan



Técnicas de creatividad

- Brainstorming
- Provocación
- SCAMPER (Sustituir, Combinar, Adaptar, Modificar, Poner otro uso, Eliminar y Reorganizar)
- Design Thinking (empatizar, definir, idear, prototipar, validar)



Brainstorming

- **¿Qué es?**

Técnica grupal para generar muchas ideas en poco tiempo.

- **Objetivo:**

Potenciar la creatividad colectiva sin juzgar ideas.

- **Reglas clave:**

No se juzga ninguna idea

Cuanto más ideas, mejor

Se fomenta construir sobre ideas de otros

- **Aplicación práctica:**

Ej. “¿Cómo podríamos mejorar la experiencia del cliente en una tienda física?”

Provocación

- **¿Qué es?**

Técnica que busca romper patrones mentales y generar ideas disruptivas.

- **Objetivo:**

Provocar el pensamiento lateral y explorar lo absurdo o ilógico.

- **Tipos de provocaciones:**

Reversión (hacer lo opuesto)

Exageración

Sustitución

- **Aplicación práctica:**

“¿Y si los restaurantes no tuvieran cocina?”

SCAMPER

- ¿Qué es?

Técnica para mejorar o transformar productos o ideas existentes.

- **SCAMPER:**

Sustituir

Combinar

Adaptar

Modificar

Proponer otros usos

Eliminar

Reordenar

- **Aplicación práctica:**

Producto: “Botella de agua”

¿Qué pa

Design Thinking

- **¿Qué es?**

Metodología centrada en las personas para resolver problemas de forma creativa.

- **Fases principales:**

Empatizar

Definir

Idear

Prototipar

Testear

- **Enfoque:**

Iterativo, colaborativo y orientado a soluciones prácticas.

- **Aplicación práctica:**

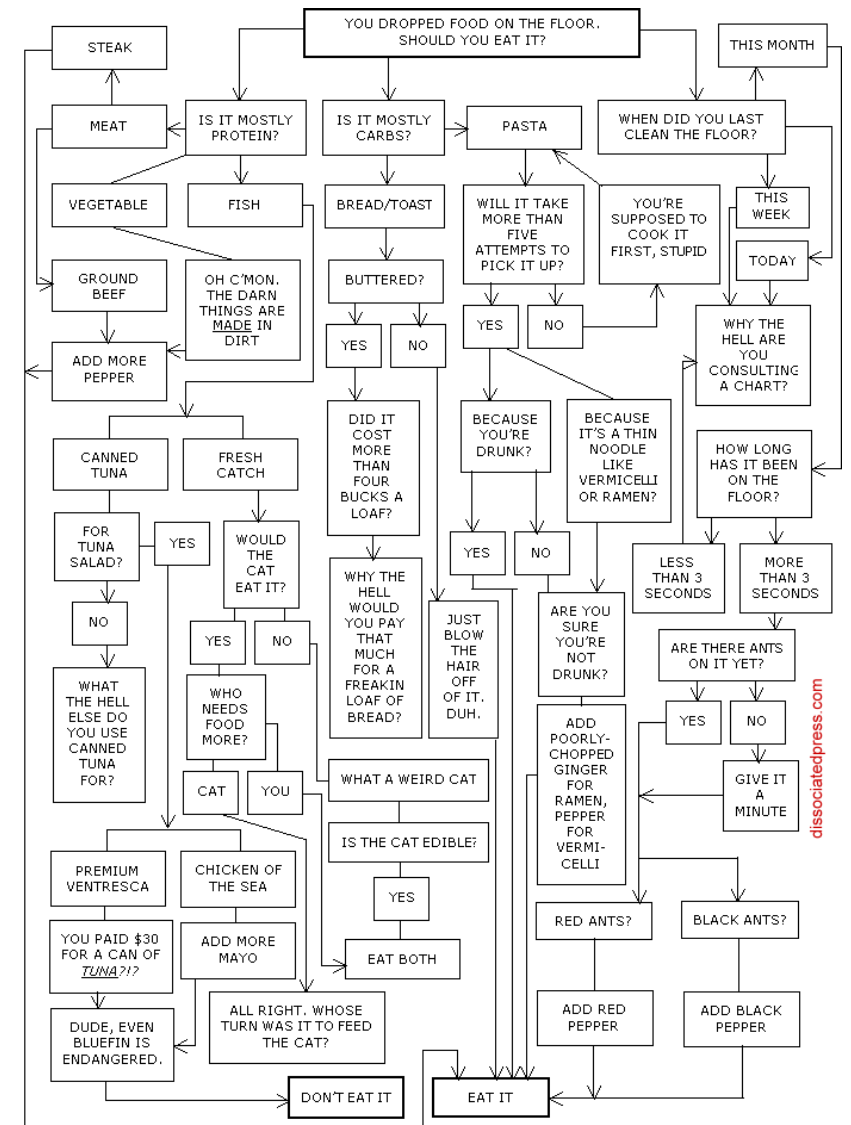
Rediseñar una silla para personas mayores → Entrevistas, bocetos, pruebas.

Análisis



Análisis

- Análisis de requerimientos
- Stakeholders
- Decidir metodología
- Tecnología necesaria
- Huir del 'technology driven'



Análisis de requerimientos

¿Qué necesita realmente el proyecto?

- Requisitos funcionales: lo que el sistema debe hacer (funciones, procesos).
- Requisitos no funcionales: rendimiento, seguridad, escalabilidad, usabilidad.
- Herramientas: entrevistas, encuestas, talleres, observación.

Buenas prácticas

- Validar con usuarios reales.
- Priorizar necesidades:
 - MoSCoW: Must have, Should have, Could have, Won't have (by the momento)
 - Kano: Analiza como se sentirá el usuario con cada funcionalidad (básica, deseada, atractiva, indiferente, reversa)
- Diferenciar deseo de necesidad.

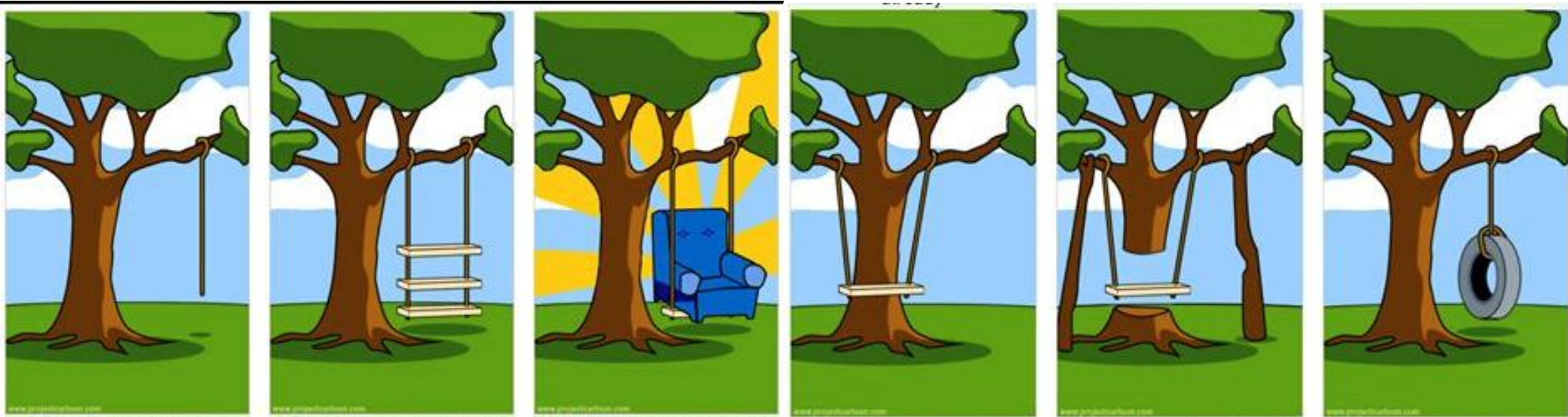
“El mayor riesgo no es no construir el sistema correctamente, sino construir el sistema equivocado.”

Toma de Requerimientos

Lo que dice que quiere

Lo que entrega el equipo de desarrollo

Lo que realmente necesita el cliente



Lo que tiene el cliente

Cómo lo describe el vendedor

Cómo lo arregla el consultor

Stakeholders

¿Quién influye y a quién afecta?

- Identificar: clientes, usuarios, patrocinadores, reguladores, etc.
- Gestionar expectativas: comunicación, transparencia, involucración.
- Evitar sesgos: no todos los stakeholders son igual de ruidosos, pero algunos silenciosos son críticos.
- Clasificar: poder vs. interés (matriz de stakeholders).

	Alto poder	Bajo poder
Alto interés	 Gestionar de cerca	 Mantener informado
Bajo interés	 Mantener satisfecho	 Monitorizar (mínimo)

“Ignorar a un stakeholder clave es como dejar una bomba sin desactivar en tu proyecto.”

Elegir el camino, no solo el destino

- ¿Qué nivel de incertidumbre tiene el proyecto?
 - Alta → Ágil
 - Baja → Predictiva
 - Mixta → Híbrida
- Criterios: tamaño del equipo, cultura organizativa, tipo de cliente.
- No casarse con una etiqueta: adaptar es mejor que dogmatizar.

	Ágil	Cascada
Flexibilidad	Alta	Baja
Entregas	Frecuentes	Final
Visibilidad	Continua	Al final

“La metodología no es el objetivo, es el vehículo.”

Tecnología: herramienta, no protagonista

- Decidir en función del problema, no de la moda.
- Evaluar:
 - ¿Soporta nuestros requerimientos?
 - ¿Nuestro equipo la domina?
 - ¿Tiene comunidad / soporte / escalabilidad?
- Evitar el síndrome del objeto brillante: “No necesitas blockchain para hacer una lista de tareas”..

Huir del ‘Technology Driven’

“Quiero usar IA aunque no la necesite.”

Metodologías II



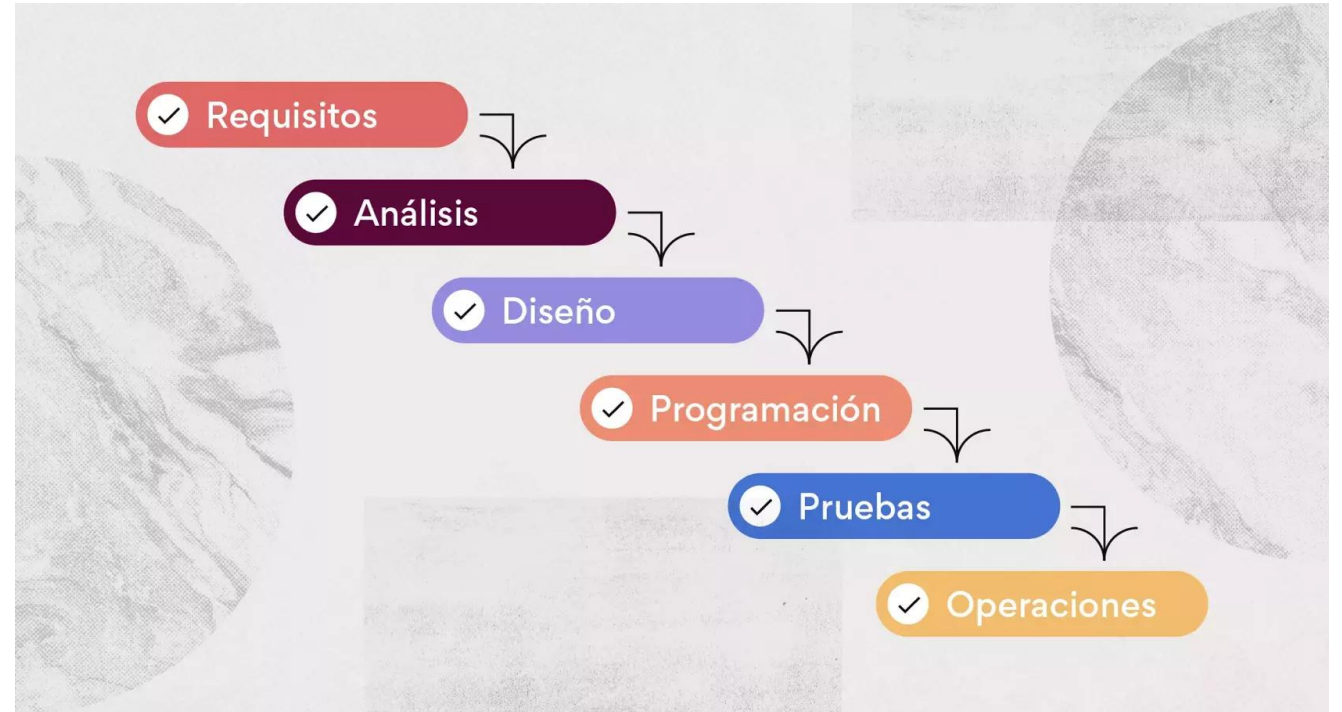
¿Qué metodologías existen y en qué se diferencian?

Metodología	Enfoque	Cuándo usarla	Ejemplo típico
Cascada	Lineal, secuencial	Requisitos claros y estables	Construcción de infraestructuras
Ágil (Scrum, Kanban...)	Iterativa, flexible	Alta incertidumbre, necesidad de adaptación	Desarrollo software
Híbrida	Mixta	Combinar control y flexibilidad	Proyectos empresariales grandes
Lean	Eficiencia	Optimización de recursos, mejora continua	Startups, producción
PMBOK	Marco tradicional	Gestión formal de grandes proyectos complejos	Ingeniería, banca, consultoría
PRINCE2	Control por fases	Ambientes regulados, foco en control	Gobierno, salud pública

“Elegir bien la metodología no garantiza el éxito... pero elegir mal puede garantizar el fracaso.”

Waterfall

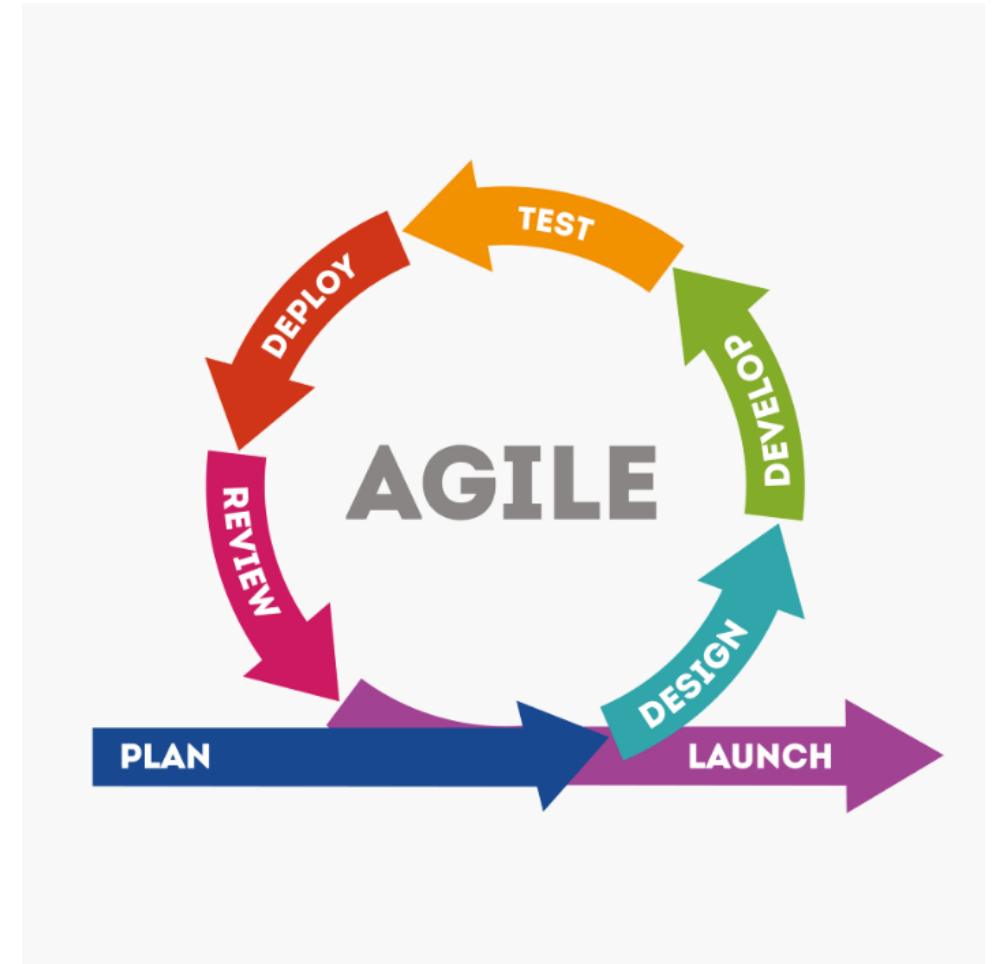
- Sencilla
- Poco flexible
- Gran empresa
- Admon. pública



Agile

- Planificación adaptativa
- Desarrollo evolutivo
- Entrega temprana
- Mejora continua
- Respuesta a los cambios

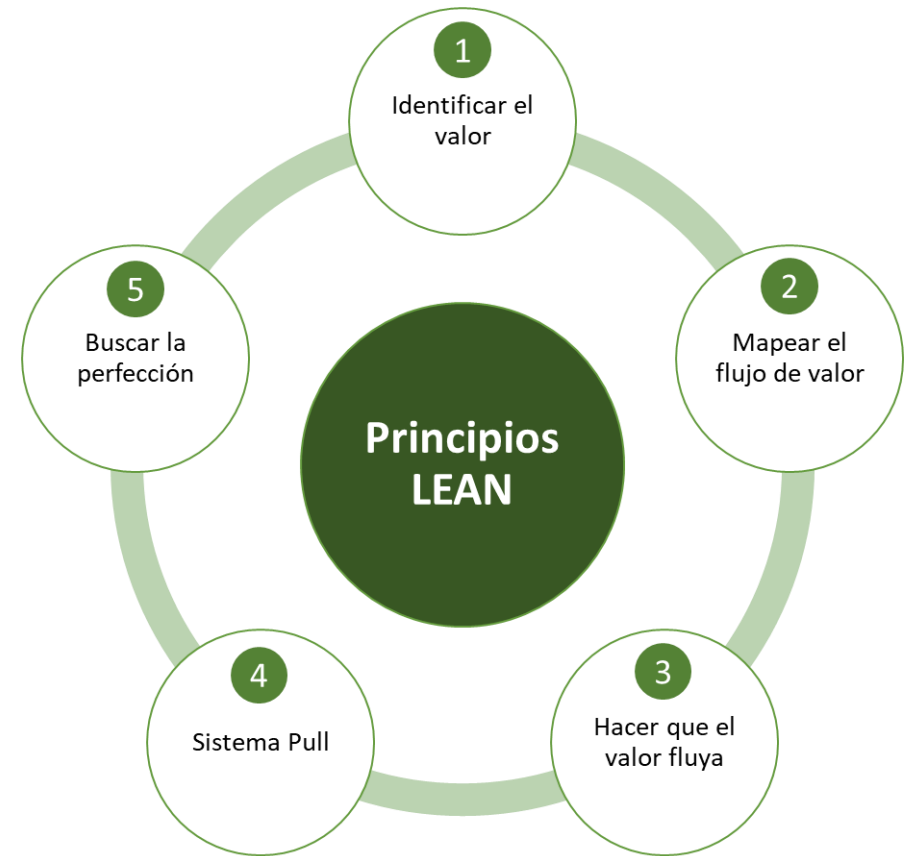
El Manifiesto Agile establece los valores fundamentales de Agile, que incluyen priorizar a las personas y las interacciones sobre los procesos y las herramientas, el software funcional sobre la documentación exhaustiva, la colaboración con los clientes sobre la negociación de contratos, y la respuesta a los cambios sobre seguir un plan estricto.



Lean

Busca optimizar los procesos de gestión y productivos. Su máxima está en reducir la inversión, el tiempo y el esfuerzo.

- Identificar el valor
- Mapear el flujo de trabajo
- Mantener un flujo continuo
- Pull, solo se inician tareas demandadas
- Búsqueda de mejora continua



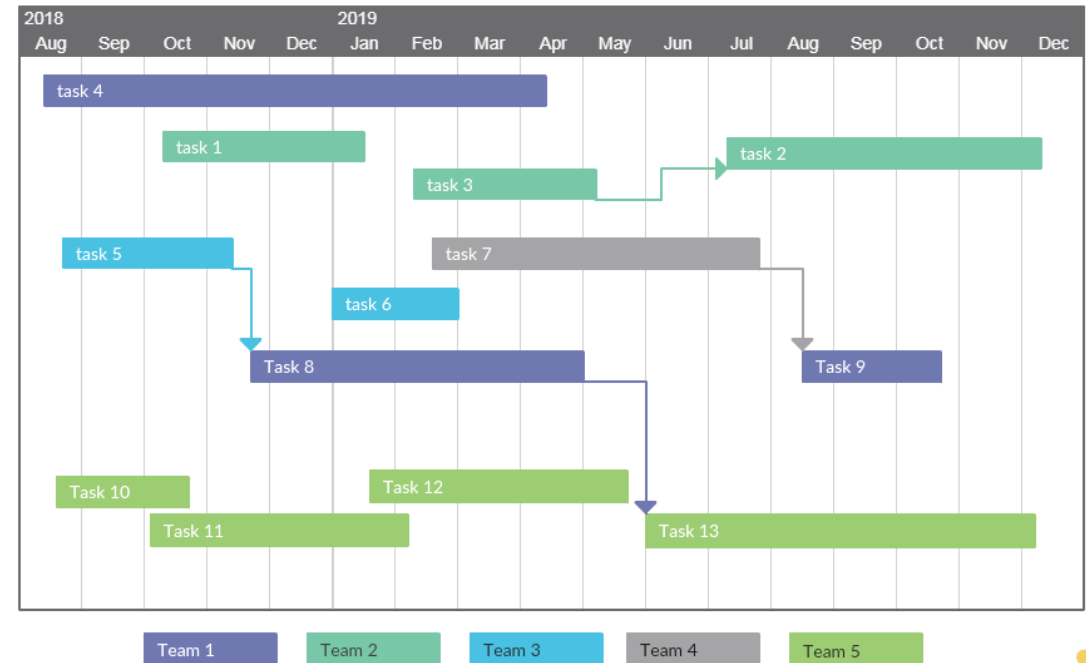
Planificación



Planificación

- Estructura del proyecto
 - Workstreams
 - Milestones
 - Cronograma
 - Caminos críticos
- Recursos, presupuesto y responsabilidades
- KPIs y mecanismos de control
- Plan de riesgos y mantenimiento
- Comunicación y reporting

High Level Gantt Chart | Project XYZ



An aerial photograph of a bridge under construction over a deep blue ocean. Several workers in orange safety vests and hard hats are positioned on the bridge deck. A white SUV is parked on the right side of the bridge. Orange traffic cones are placed along the edge of the bridge deck. The bridge structure is made of concrete and steel, with railings visible. The water is a vibrant blue, and the sky is not visible.

¿Por qué es importante
una buena planificación?

Herramientas

- Jira/Atlassian
- Trello
- Linear
- Active Collab
- GitHub
- Office



Definition of Done



Definition of Done (DoD)

Es un acuerdo del equipo sobre que significa que algo esta terminado

Sin DoD, cada persona tiene un criterio diferente de 'hecho'

Ejemplo de DoD para una historia de usuario:

- Codigo escrito y revisado (code review aprobado)

- Tests unitarios escritos y pasando

- Tests de integracion pasando

- Documentacion actualizada (si aplica)

- Desplegado en entorno de test

- QA ha validado los criterios de aceptacion

- Sin bugs criticos abiertos

Criterios de aceptacion vs Definition of Done

Son cosas diferentes y complementarias:

Criterios de aceptacion: especificos de cada historia

'Como usuario, puedo canjear puntos y mi saldo se reduce correctamente'

Definition of Done: comun a TODAS las historias

'El codigo tiene tests, esta revisado y desplegado en test'

Una historia esta terminada cuando cumple SUS criterios de aceptacion

Y ADEMÁS cumple la Definition of Done del equipo

Roles en el equipo



Roles tipicos en un proyecto agil

Product Owner (PO): representa al negocio, prioriza el backlog, decide que se construye

No es el jefe del equipo: es la voz del cliente dentro del equipo

Scrum Master / Facilitador: elimina impedimentos, facilita ceremonias, protege al equipo

No es un gestor de proyecto tradicional: no asigna tareas

Tech Lead: decisiones tecnicas, arquitectura, revision de codigo

Desarrolladores: construyen el producto

QA / Tester: valida que lo construido cumple los criterios de aceptacion

DevOps: infraestructura, CI/CD, despliegues

Antipatrones de roles

El PO que no esta disponible: el equipo toma decisiones sin el y luego hay que rehacer

El Tech Lead que decide todo: el equipo no aprende ni aporta

El Scrum Master que es jefe encubierto: asigna tareas y controla, no facilita

No tener QA: 'ya lo probamos nosotros' = bugs en produccion

Todo el mundo es 'full-stack': nadie es responsable de nada concreto

La clave: roles claros con responsabilidades explicitas desde el dia 1

Ejecución



Ejecución

- Desarrollo
- Seguimiento de KPIs
- Integración de workstreams
- Test plan
- Comunicación y coordinación
- Steering comitee



Continuar y Pivotar

- Es importante conocer la salud del proyecto, los posibles cambios en requerimientos y tomar decisiones en cada workstream sobre continuar o pivotar.



MVP Producto Mínimo Viable



Entrega algo pequeño que funcione, valida, itera

¿Que es un MVP?

Minimum Viable Product: la version mas pequena del producto que aporta valor real

No es un prototipo a medias. Es un producto funcional, solo que con alcance reducido

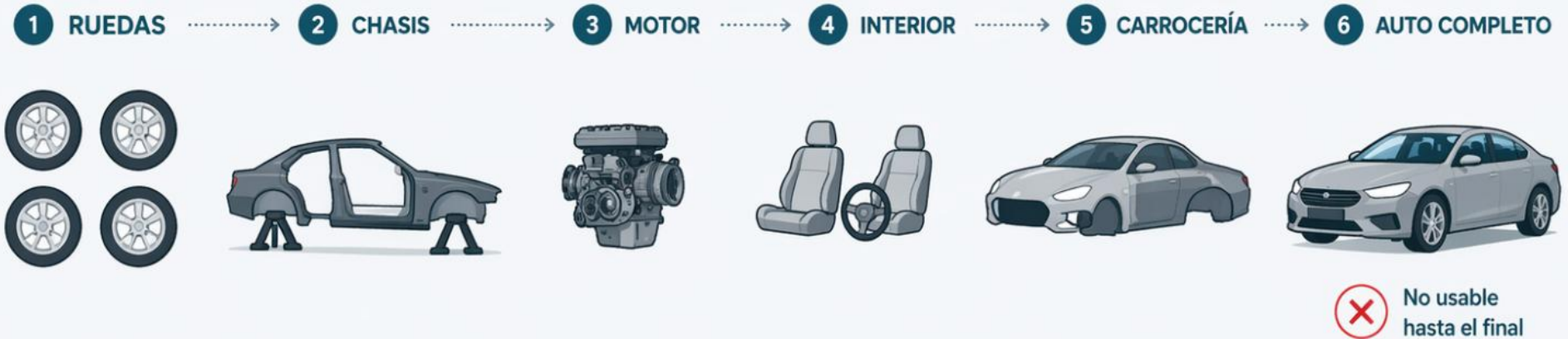
El objetivo: validar hipotesis con usuarios reales lo antes posible

Si esperas a tener el producto completo para validar, llegas tarde

MVP: lo que SI es vs lo que NO es

NOT an MVP

Construir
todo de una vez.
No usable hasta
el final.



MVP approach

Evolucionar en
pasos. Usable
desde el inicio.



MVP en practica: como definirlo

1. Listar TODAS las funcionalidades deseadas
2. Clasificar: Must have / Should have / Could have / Won't have (MoSCoW)
3. El MVP = solo los Must have
4. Entregar, poner delante del usuario, recoger feedback
5. Iterar: anadir Should have segun el feedback

Ejemplo FidelityChain:

Must: registrar cliente, emitir puntos, canjear, consultar saldo

Should: historial, info del token, catalogo de productos

Could: eventos en tiempo real, web UI, multiples partners

Won't (v1): caducidad de puntos, transferencia entre clientes

Entrega



Entrega

- Pruebas de aceptación
- Formación a usuarios
- Mantenimiento
- Evaluación de la satisfacción



Riesgos



¿Que es un riesgo?

Un riesgo es un evento incierto que, si ocurre, tiene un impacto en el proyecto

No es un problema: un problema ya ha ocurrido. Un riesgo PUEDE ocurrir

Todo proyecto tiene riesgos. La diferencia esta en si los gestionas o te pillan por sorpresa

Dos dimensiones:

Probabilidad: ¿que tan probable es que ocurra? (alta/media/baja)

Impacto: si ocurre, ¿como de grave es? (critico/alto/medio/bajo)

Riesgo = Probabilidad x Impacto

Matriz de riesgos: ejemplo para un proyecto

Riesgo	Probabilidad	Impacto	Mitigacion
Un miembro del equipo deja el proyecto	Media	Alto	Documentar todo, pair programming
La tecnologia elegida no soporta un requisito	Baja	Critico	PoC temprana antes de comprometerse
El cliente cambia los requisitos a mitad	Alta	Medio	Contratos agiles, MVP, entregas frecuentes
Fallo de seguridad en produccion	Baja	Critico	Auditorias, tests de seguridad, pen testing
Retraso en la integracion con sistema externo	Media	Alto	Mocks, interfaces desacopladas

Plan de respuesta a riesgos

Para cada riesgo identificado, elegir UNA estrategia:

Evitar: eliminar la causa del riesgo (cambiar el enfoque, reducir alcance)

Mitigar: reducir la probabilidad o el impacto (PoC, tests, documentacion)

Transferir: pasar el riesgo a otro (seguros, subcontratar, SLAs)

Aceptar: asumir que puede pasar y tener un plan B preparado

Los riesgos se revisan periodicamente. No es un ejercicio de una sola vez.

Particularidades Blockchain



¿Que tiene de especial un proyecto blockchain?

En un proyecto tradicional hay UN cliente y UN equipo

En un proyecto blockchain enterprise hay un CONSORCIO:

Multiples organizaciones que son a la vez clientes Y participantes

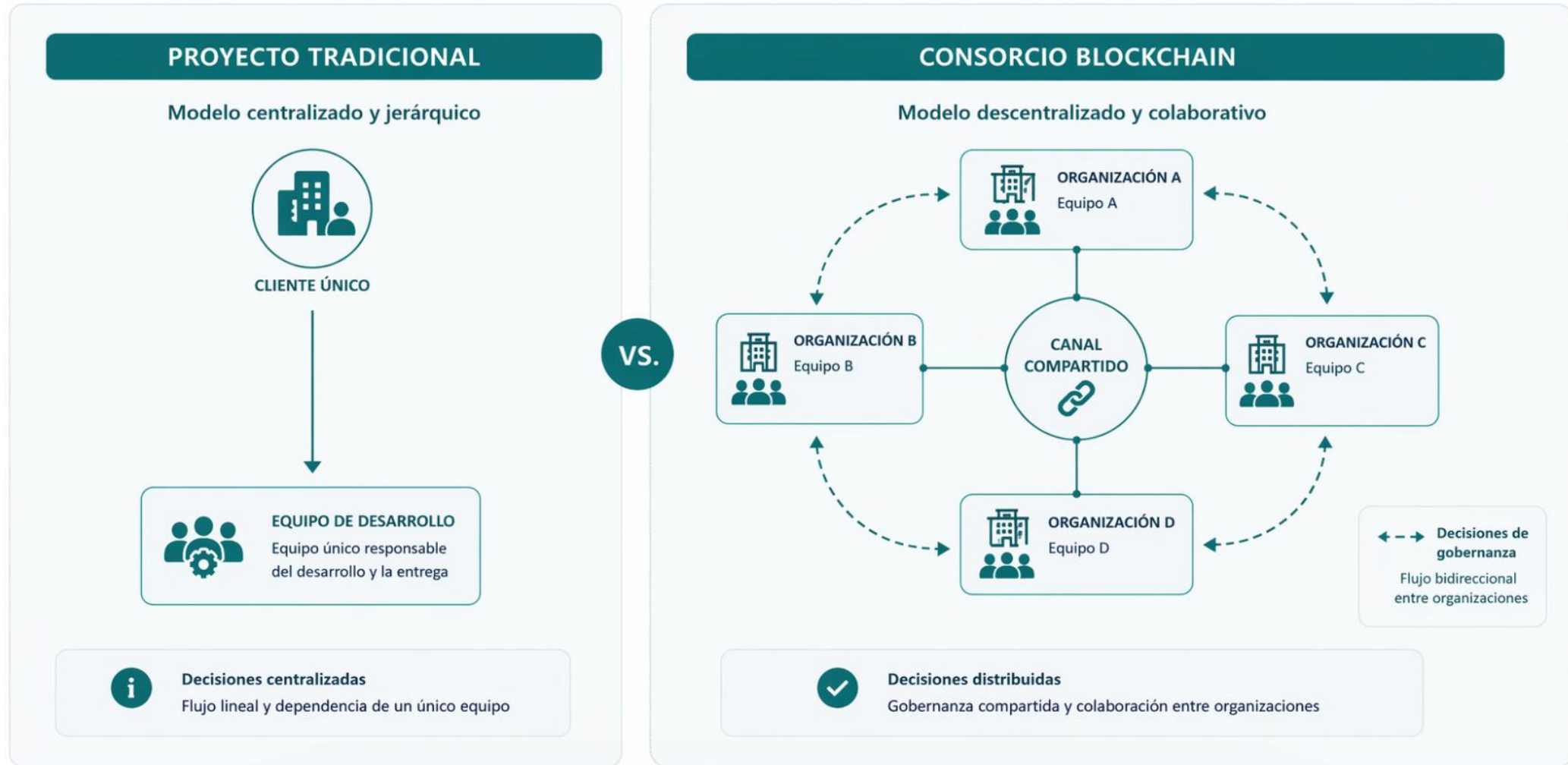
Cada org tiene sus propios intereses, plazos y prioridades

Las decisiones tecnicas afectan a todas las orgs (politicas de endorsement, datos compartidos)

El despliegue requiere coordinacion entre orgs (lifecycle: approve de cada una)

Gestionar un proyecto blockchain es gestionar un proyecto POLITICO ademas de tecnico

Proyecto tradicional vs proyecto blockchain



La gobernanza como workstream

En un proyecto Fabric, la gobernanza del consorcio NO es un detalle tecnico
Es un workstream completo que hay que planificar desde el dia 1:

- ¿Quien puede proponer cambios en el chaincode?
- ¿Como se aprueba un nuevo chaincode o una actualizacion? (lifecycle)
- ¿Quien paga la infraestructura? (orderers, CAs)
- ¿Como se incorpora una nueva organizacion al consorcio?
- ¿Que pasa si una organizacion quiere salir?
- ¿Quien tiene acceso a que datos? (canales, Private Data)

Si no resuelves esto antes de escribir codigo, el proyecto fracasara.
TradeLens y B3i fracasaron por gobernanza, no por tecnologia.

Coordinacion de despliegues

En un proyecto normal: despliegas en TU servidor y ya esta

En Fabric: el despliegue requiere acciones de CADA organizacion

Para desplegar un chaincode:

1. Alguien empaqueta el codigo y lo distribuye a las orgs
2. Cada org lo revisa, lo instala en SU peer y lo aprueba
3. Solo cuando la mayoria ha aprobado se puede hacer commit

Esto significa: coordinacion de agendas, ventanas de despliegue, planes de rollback consensuados y comunicacion constante

Consejo: automatizar al maximo con scripts y CI/CD

Riesgos especificos de proyectos blockchain

Adopcion: que las orgs del consorcio no se comprometan o abandonen a mitad

Fundador dominante: una org controla demasiado y las demas desconfian

Complejidad de integracion: conectar Fabric con los sistemas legacy de cada org

Regulacion cambiante: leyes de proteccion de datos, smart contracts como contratos legales

Rendimiento: expectativas irreales de TPS o latencia

Seguridad: un certificado comprometido de una org afecta a toda la red

Mitigacion clave: empezar con un MVP muy pequeno entre 2-3 orgs

y ampliar solo cuando se demuestre valor.

El MVP en blockchain: empieza muy pequeño

Error tipico: intentar montar un consorcio de 20 orgs desde el dia 1

Enfoque correcto: 2-3 orgs, 1 canal, 1 chaincode sencillo

Fase 1 (MVP): 2 orgs, caso de uso basico, red minima

Ejemplo: Hotel + Cafeteria con puntos de fidelizacion

Objetivo: demostrar que funciona y que aporta valor

Fase 2: anadir 1-2 orgs mas, ampliar funcionalidades

Ejemplo: anadir un restaurante y una tienda al programa de puntos

Fase 3: escalar a produccion, governance formal, SLAs

Solo si la Fase 1 y 2 demostraron valor real

1. ¿Que es mas dificil: la tecnologia o la gobernanza del consorcio? ¿Por que?
2. Si tuvierais que convencer a 3 empresas de vuestro sector para formar un consorcio, ¿que argumentos usariais?
3. ¿Como gestionariais el riesgo de que una org abandone el consorcio a mitad del proyecto?
4. ¿MVP de 2 orgs o lanzamiento con todo el consorcio? ¿Que preferis y por que?
5. ¿Que ceremonias agiles adaptariais para un proyecto multi-organizacion?